

Documentation Technique – POC DataPress

Auteur : Lucas **Date :** Novembre 2025 **Objectif :** Démonstration d'une architecture microservices conteneurisée et de bonnes pratiques DevOps.

1. Introduction et Contexte du Projet

Le projet DataPress est un Proof of Concept (POC) visant à illustrer l'implémentation d'une architecture orientée microservices. L'objectif principal est de valider les méthodes modernes de déploiement et d'opération, en mettant l'accent sur la modularité, la robustesse et la facilité de maintenance.

Objectifs Clés du POC :

- **Démonstration Pédagogique :** Présenter une architecture microservices complète et fonctionnelle.
- **DevOps Pratiques :** Mettre en œuvre les standards de conteneurisation (Docker) et d'orchestration (Kubernetes).
- **Valeur Ajoutée :** Fournir une base technique maintenable, évolutive, et facilement industrialisable. L'architecture est conçue pour supporter une intégration continue et un déploiement continu (CI/CD) dès le départ.

2. Architecture des Microservices

L'application est composée de trois services indépendants, chacun encapsulé dans son propre conteneur Docker. Cette séparation garantit une isolation des dépendances et des responsabilités.

2.1. Détail des Services

Service Technologie		Rôle	Port Interne
API	Flask + Gunicorn	Service métier principal, expose les données via des endpoints RESTful.	5000
Front	NGINX	Serveur web statique et Reverse Proxy, point d'entrée utilisateur.	80
Auth	Flask	Service optionnel dédié à la gestion d'identité et des jetons d'accès.	5001

2.2. Flux Réseau et Isolation

La communication est stricte et isolée. Les services internes communiquent via le réseau Docker nommé datapress-net. Seuls les ports nécessaires sont exposés : 80 (Front), 5000 (API) et 5001 (Auth) à l'hôte, ou via un mécanisme d'équilibrage de charge en production (LoadBalancer/Ingress).

2.3. Description du Flux de Communication

1. **Requête Client** : Le client accède à l'application via le port 80 (géré par le conteneur **Front** NGINX).
2. **Routage Statique** : NGINX sert les actifs statiques (HTML, CSS, JS).
3. **Appel API** : Pour toute donnée dynamique, NGINX agit comme un **proxy inverse** et transmet la requête au service **API** via le réseau interne datapress-net sur le port 5000.
4. **Vérification d'Authentification (Optionnel)** : L'**API** peut interroger le service **Auth** sur le port 5001 (toujours via le réseau interne isolé) pour valider les informations d'identification de l'utilisateur, garantissant ainsi que l'Auth Service n'est jamais directement exposé à l'extérieur.

3. Conteneurisation (Docker)

La conteneurisation assure l'immuabilité de l'infrastructure et la reproductibilité de l'environnement de développement à la production.

3.1. Bonnes Pratiques de Build

Chaque service utilise un Dockerfile dédié et optimisé, adhérant aux standards de l'industrie :

- **Multi-stage Build (API)** : Cette technique permet de réduire l'image finale en n'incluant que les artefacts d'exécution (le code, les dépendances runtime), excluant ainsi les outils de développement et de build, ce qui est crucial pour la sécurité et la performance.
- **Utilisateur Non-Root** : L'exécution des conteneurs s'effectue avec un utilisateur  **non-root** pour limiter les privilèges. En cas de compromission, l'attaquant ne dispose pas des droits d'administrateur sur le conteneur ou l'hôte Docker.
- **Serveur WSGI Production** : Pour l'API Python, **Gunicorn** est utilisé comme serveur WSGI de production, assurant la gestion des processus, la concurrence et la stabilité, ce qui est supérieur à l'utilisation du serveur de développement Flask intégré.
- **Images Légères** : L'utilisation de bases légères comme python:3.11-slim et nginx:alpine garantit des temps de téléchargement rapides et une réduction de la surface d'attaque due à moins de paquets inutiles.

3.2. Orchestration Locale avec Docker Compose

Le fichier docker-compose.yml est le manifeste de l'environnement de développement et de test local.

Élément	Fonction	Détail Technique
Services	Définition des conteneurs	Image, ports, variables d'environnement, et la configuration du volume.
Dépendances	depends_on	Assure l'ordre de démarrage initial des services (ex: l'API attend le démarrage d'une base de données simulée).
Réseau	datapress-net	Assure l'isolation et une résolution de noms DNS interne stable (ex: l'API trouve l'Auth service via son nom de service).
Logs	Standard Out/Err	Les journaux sont collectés de manière uniforme et unifiée via la commande docker-compose logs.

4. Déploiement en Cluster (Kubernetes)

Le projet utilise des manifests K8s standardisés dans le répertoire k8s/ pour préparer le déploiement sur tout cluster compatible.

Objets Kubernetes Utilisés et leurs Rôles :

- **Deployment** : Gère l'état désiré de l'application (le nombre de replicas pour le *scaling*) et orchestre les mises à jour progressives (*rolling updates*) et les *rollbacks* sans interruption de service.
- **Service** : Fournit une adresse IP stable et un nom DNS interne pour accéder aux pods du Deployment. Il garantit que le Front peut toujours atteindre l'API, quel que soit le pod spécifique.
- **ConfigMap** : Stocke la configuration non-sensible (URLs d'API, chemins de fichiers, etc.), permettant de découpler la configuration du code de l'image.
- **Secret** : Stocke les informations sensibles (clés d'API, mots de passe de base de données, jetons), injectées dans les pods sous forme de variables d'environnement ou de volumes.

Note sur le Déploiement en Cluster : Sur un cluster (contrairement au local), l'exposition publique se fait via un objet **Ingress** (pour le routage HTTP/HTTPS basé sur le nom d'hôte) ou un **LoadBalancer** (pour l'exposition au niveau réseau). Ces mécanismes remplacent l'exposition directe des ports sur l'hôte utilisée par Docker Compose.

5. Exploitation et Observabilité

5.1. Commandes d'Exploitation Rapide

Action	Commande	Résultat Attendu
Démarrage/Build	<code>docker-compose up --build</code>	Déploiement, construction des images locales et lancement de tous les services.
Arrêt & Nettoyage	<code>docker-compose down --volumes</code>	Arrêt des conteneurs et suppression des réseaux et des volumes associés (nettoyage complet).
Suivi des Logs	<code>docker-compose logs -f</code>	Affichage en temps réel des logs agrégés de tous les services.
Test API	<code>curl http://localhost:5000/health</code>	Réponse HTTP 200 (OK). Le conteneur répond correctement et le processus Unicorn est sain.

5.2. Supervision

- **Healthchecks Intégrés** : L'endpoint `/health` est vital. Il est utilisé comme LivenessProbe et ReadinessProbe dans Kubernetes pour :
 - **Liveness** : Déterminer si un pod doit être redémarré en cas de défaillance (processus figé).
 - **Readiness** : Déterminer si un pod est prêt à recevoir du trafic réseau.
- **Extensions Futures pour l'Observabilité** : Le POC est conçu pour intégrer facilement une stack d'observabilité complète :
 - **Métriques (Prometheus)** : Collecte des données temporelles sur la performance des pods (CPU, mémoire, I/O) et des métriques applicatives (latence des endpoints, taux d'erreurs 5xx).
 - **Visualisation (Grafana)** : Création de tableaux de bord pour l'analyse des tendances et la détection proactive des anomalies.
 - **Traçage Distribué** : Mise en place d'un mécanisme pour suivre le parcours d'une requête à travers l'ensemble des microservices (API -> Auth -> Base de données, etc.).

6. Bonnes Pratiques Synthétisées

1. **Microservices** : Clarté et séparation des responsabilités, permettant des mises à jour indépendantes.
2. **Immuabilité** : Les conteneurs ne sont pas modifiés après le build ; les configurations externes passent par ConfigMap/Secret.
3. **Sécurité** : Utilisation d'utilisateur non-root, exposition minimale des ports, et gestion sécurisée des Secrets.
4. **Optimisation** : Images de conteneurs légères et builds rapides pour une efficacité maximale.
5. **Automatisation** : Préparation complète de l'architecture pour l'intégration et le déploiement continu (CI/CD).

7. Conclusion

Le POC DataPress démontre avec succès la mise en place d'une architecture microservices conteneurisée, sécurisée et supervisée. Il fournit une base technique robuste pour tout projet client nécessitant évolutivité, industrialisation et adhésion aux meilleures pratiques DevOps.